

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _Narmatha J_

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

ANSWERS:

1. Hadoop Job Failure Due to Uneven Input Splits

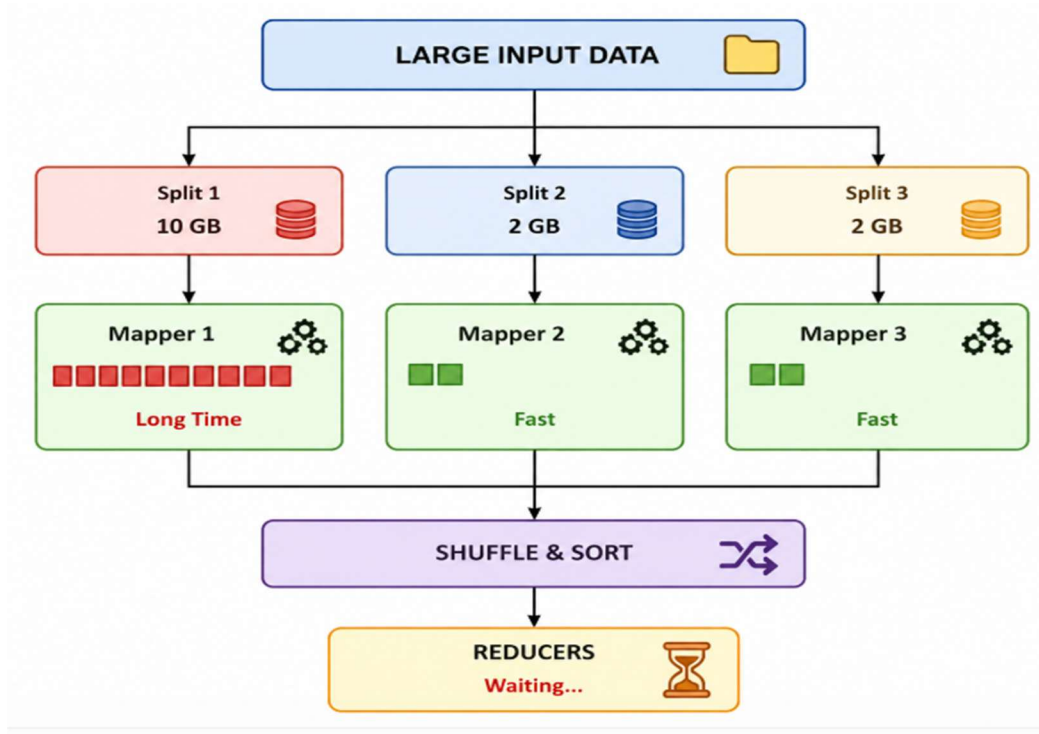
Problem Identification:

The Hadoop job repeatedly fails because the input data is not divided equally among the Mapper tasks. Some input splits contain much more data than others. Therefore, some Mappers take a long time to complete while other Mappers finish quickly. Since Reducers receive data only after the Mapper phase produces intermediate results, the Reducers remain idle for a significant amount of time.

Likely Design Error:

The main design error is improper input partitioning and data imbalance. Hadoop works efficiently when the input data is divided into reasonably balanced blocks. If one split is much larger than the others, the corresponding Mapper becomes a bottleneck.

Design Diagram:



Analysis:

Mapper 1 receives 10 GB while Mapper 2 and Mapper 3 receive only 2 GB each. Mapper 2 and Mapper 3 complete their work much earlier and remain idle. The Reducers cannot efficiently process the data until the Mapper outputs are available.

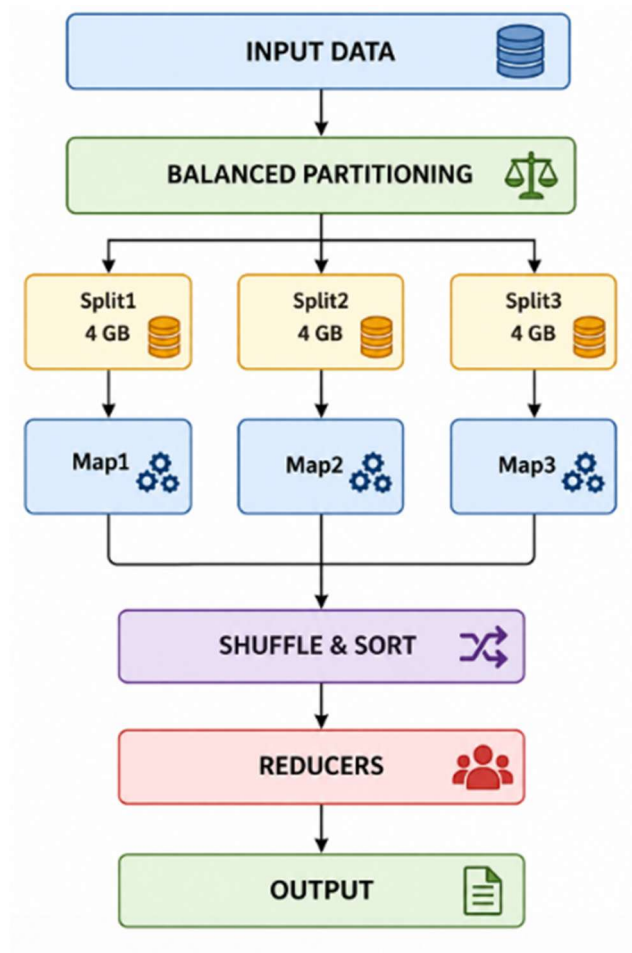
This creates:

- Uneven Mapper workload
- Longer job execution time
- Reducer idle time
- Poor cluster utilization
- Increased possibility of job timeout or failure

Corrective Actions:

1. Use suitable HDFS block size and input split configuration.
2. Divide large files into balanced input files.
3. Avoid highly skewed input data.
4. Use appropriate partitioning techniques.
5. Monitor Mapper execution time to identify slow tasks.
6. Reorganize the input dataset when severe imbalance exists.

Improved Design:



Expected Result:

Balanced input splits distribute the workload more evenly among Mappers. This keeps the Reducers supplied with data and improves overall Hadoop job performance.

Final Solution:

The input data should be properly partitioned and balanced so that no single Mapper becomes a bottleneck.

2. HDFS Data Unavailability After Node Failure

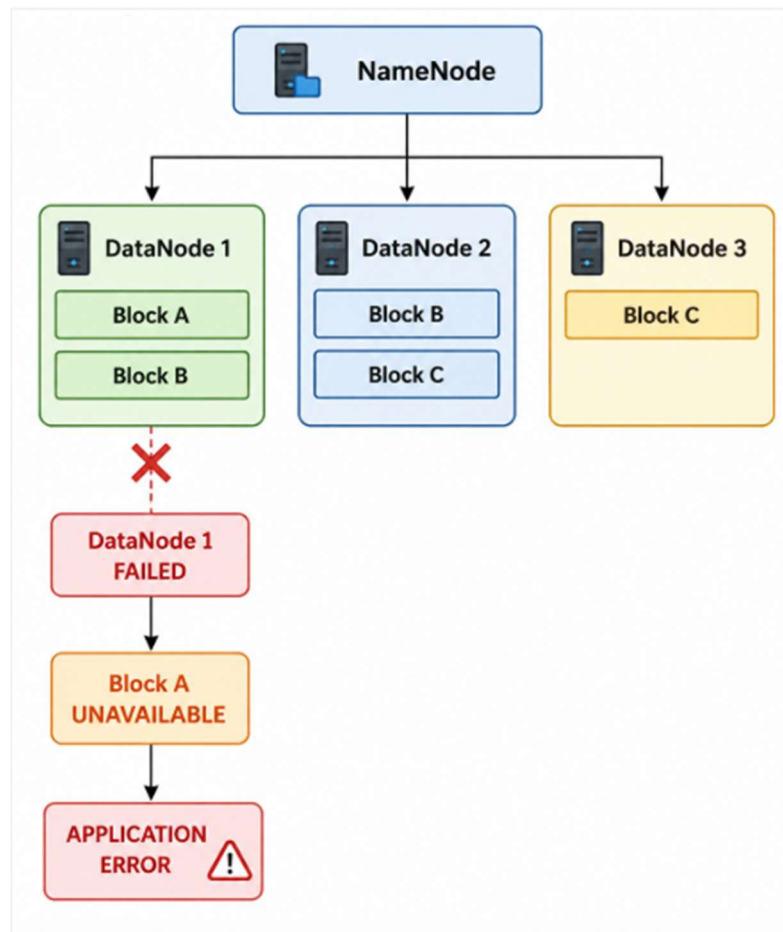
Problem Identification:

The HDFS system becomes unavailable when one DataNode fails. In a properly designed HDFS cluster, a single DataNode failure should normally not make the stored data unavailable because HDFS maintains multiple copies of data blocks.

Likely Design Error:

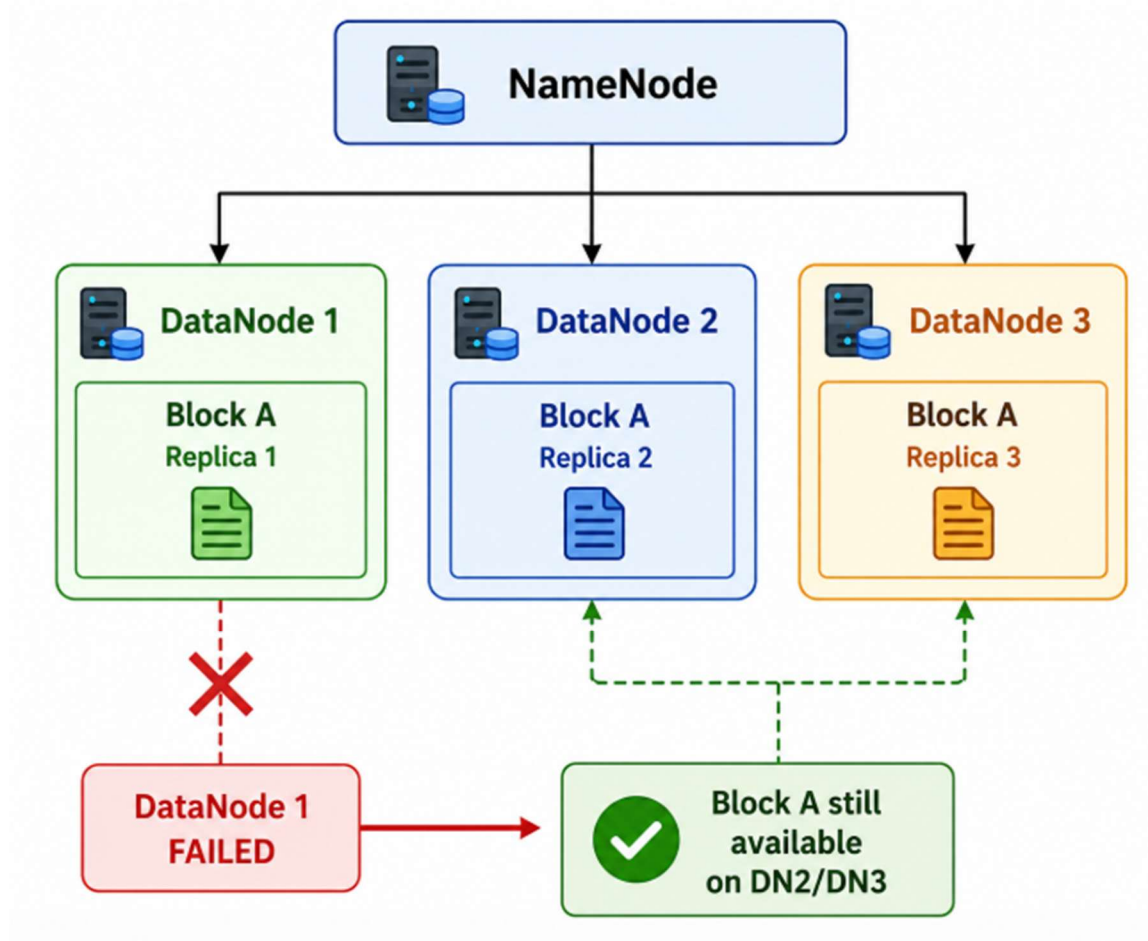
The probable error is an insufficient replication factor or incorrect placement of replicas. Another possible issue is the absence of NameNode High Availability, which can create a single point of failure.

Design Diagram – Faulty System:



Here, important blocks may exist on only one DataNode. When that DataNode fails, the corresponding data cannot be accessed.

Correct HDFS Design:



Analysis:

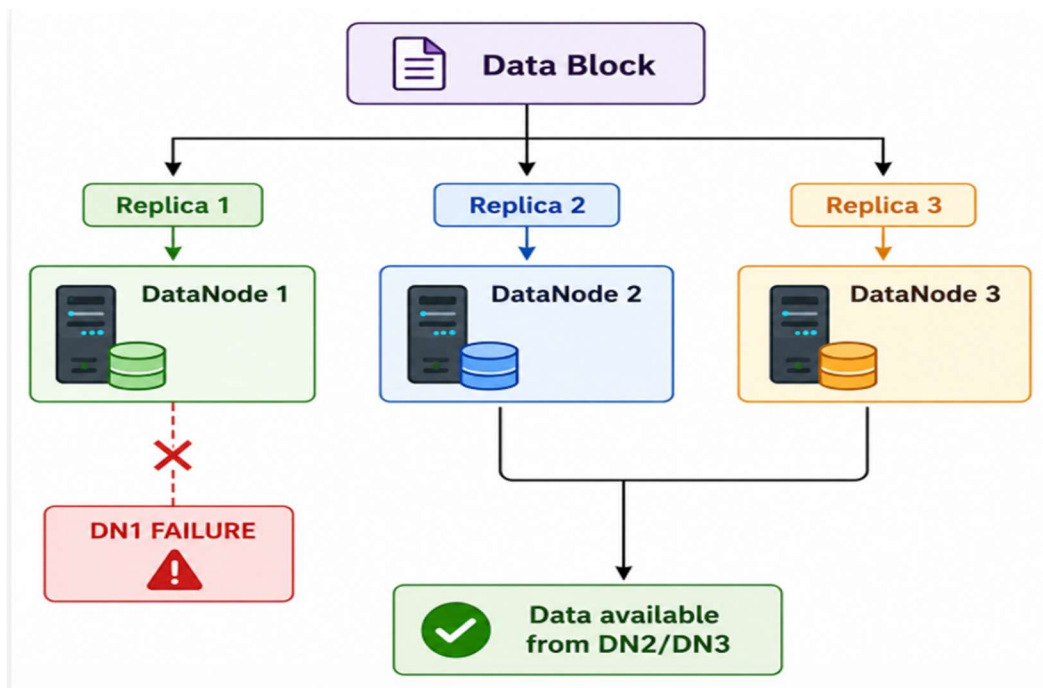
HDFS achieves fault tolerance by storing multiple replicas of each block. If the replication factor is too low, a node failure can result in data unavailability.

The NameNode is also critical because it maintains the metadata about files and blocks. If only one NameNode is used and it fails, the entire HDFS namespace may become inaccessible.

Corrective Actions:

- Maintain a suitable HDFS replication factor.
- Distribute replicas across different DataNodes.
- Use rack-aware replica placement.
- Implement NameNode High Availability.
- Maintain Active and Standby NameNodes.
- Monitor failed or under-replicated blocks.
- Regularly check HDFS health.

Expected Result:



Final Solution:

Proper block replication combined with NameNode High Availability prevents a single node failure from causing major data unavailability.

3. Apache NiFi – Duplicate Records in Analytics Store

Problem Identification:

The ETL workflow receives data through Apache NiFi and transfers it to an analytics database. However, the same records are inserted multiple times.

For example:

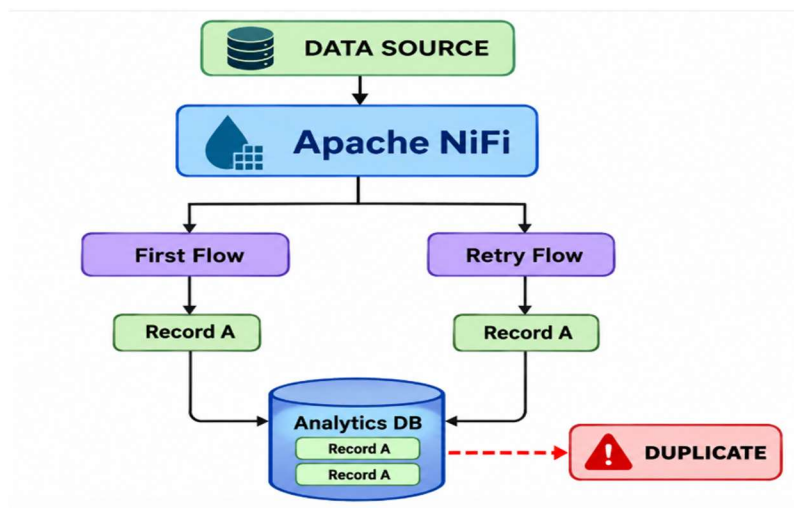
Customer ID	Name
1001	Ravi
1001	Ravi
1002	Arun

The record with ID 1001 has been inserted twice.

Likely Design Error:

The most probable error is the absence of duplicate detection, unique identification, and idempotent processing.

Faulty Design:

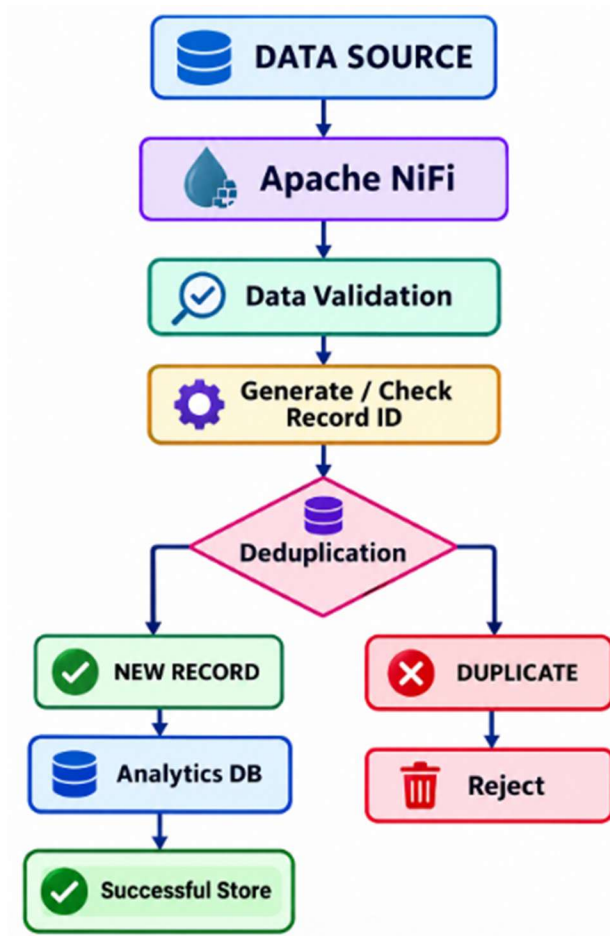


Analysis:

Duplicates can occur for several reasons:

1. The same input file may be processed more than once.
2. Network failures may cause a transaction to be retried.
3. NiFi may receive duplicate messages from the source.
4. There may be no unique record identifier.
5. The database may not have a unique constraint.
6. The ETL workflow may not perform deduplication.

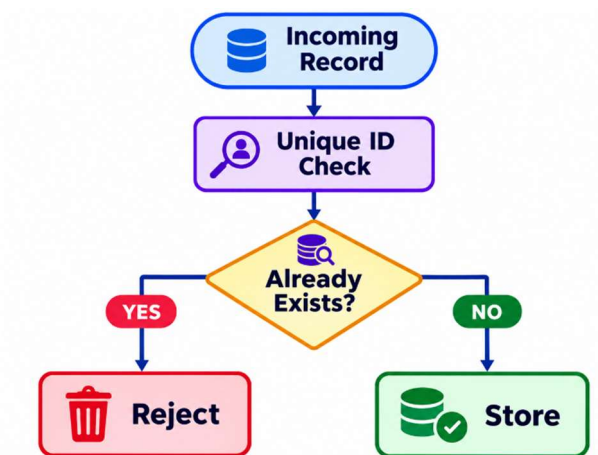
Corrected Design:



Corrective Actions:

- Assign a unique ID to every record.
- Check whether the ID already exists.
- Configure NiFi to handle duplicate FlowFiles.
- Use database unique constraints.
- Use idempotent database operations.
- Handle retries carefully.
- Validate records before inserting them.

Expected Result:



Final Solution:

A combination of NiFi deduplication + unique record identification + database constraints prevents duplicate records from reaching the analytics store.

4. YARN Resource Starvation

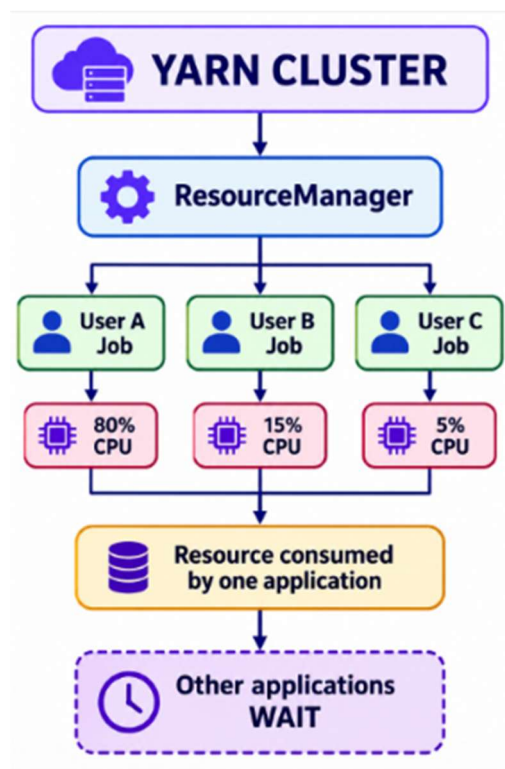
Problem Identification:

The YARN cluster receives jobs from several users at the same time. One or more jobs consume a large portion of CPU and memory, leaving insufficient resources for other jobs.

Likely Design Error:

The main design error is poor resource scheduling. The cluster does not properly divide resources among users or applications.

Faulty Design:



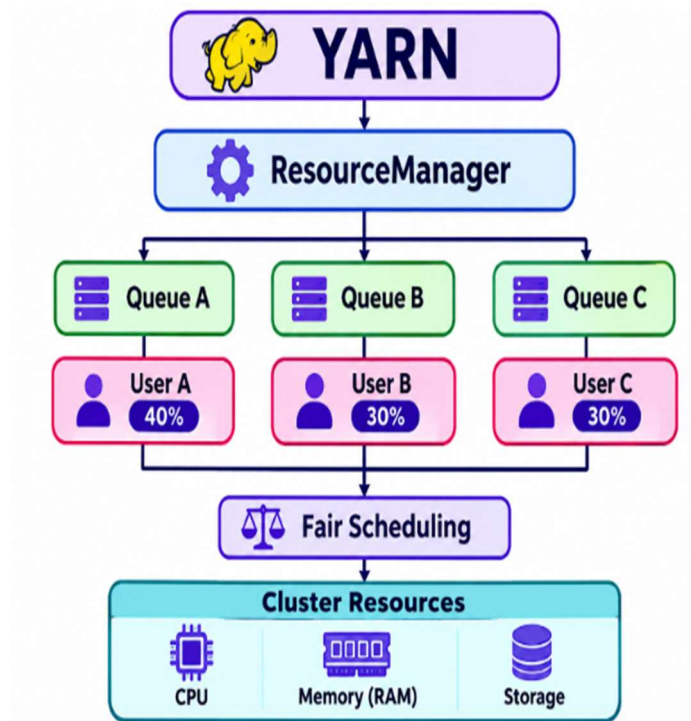
Analysis

If a single user submits a resource-intensive job, that job can consume most of the available CPU and memory. Other users then experience long waiting times.

This results in:

- Resource starvation
- Poor cluster utilization
- Long job waiting time
- Unfair resource allocation
- Reduced user satisfaction

Improved Design:



Corrective Actions:

1. Configure Capacity Scheduler or Fair Scheduler.
2. Create separate queues for different users.
3. Assign resource limits to each queue.
4. Control CPU and memory allocation.

5. Prevent a single application from consuming all resources.
6. Monitor cluster resource utilization.
7. Prioritize important jobs when necessary.

Expected Result:



Final Solution:

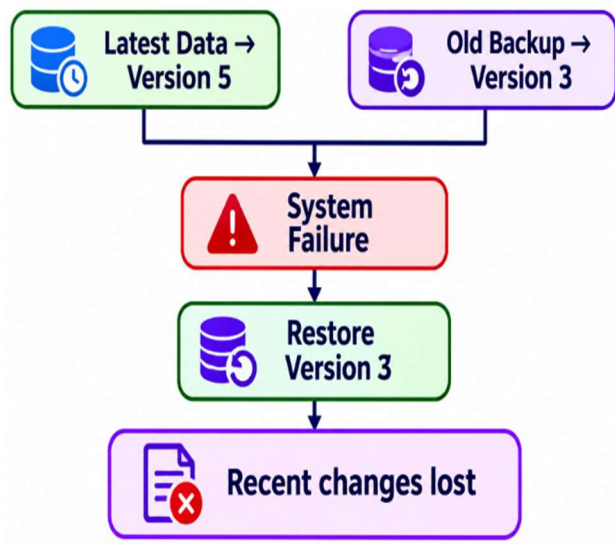
YARN should use a proper scheduler with queue-based resource allocation to ensure fair and controlled resource usage.

5. Backup Restores Outdated Data

Problem Identification:

After a failure in distributed storage, the recovery process restores an older version of the data instead of the latest available version.

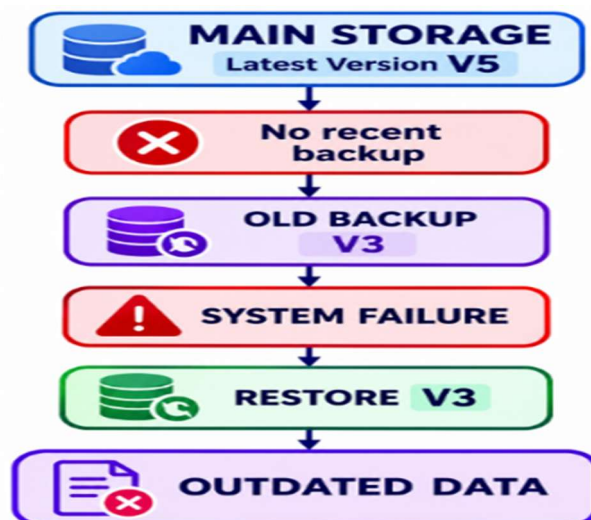
For example:



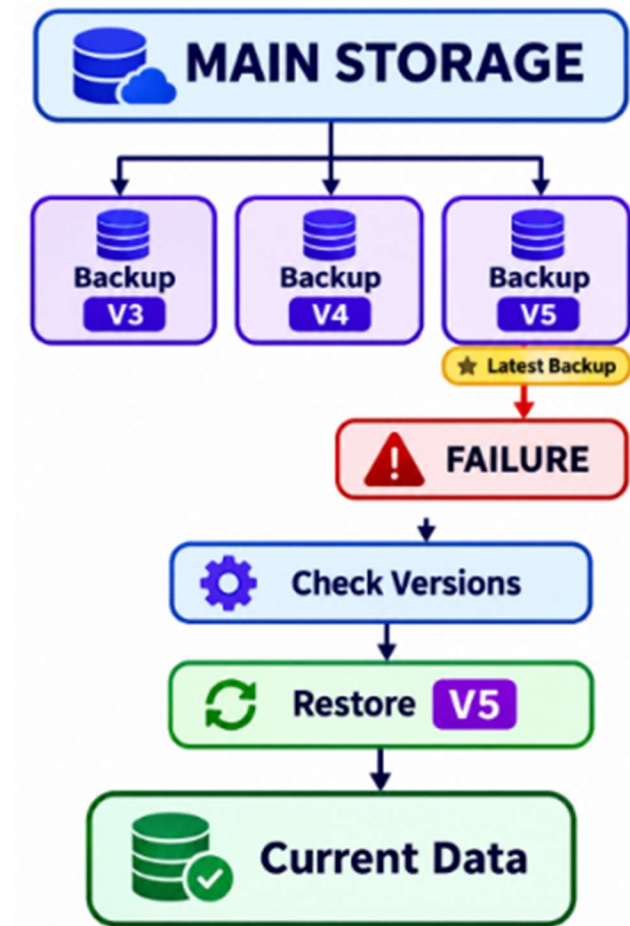
Likely Design Error:

The backup strategy is not maintaining the latest synchronized copy of the data. It may also lack proper version tracking and backup verification.

Faulty Design:



Corrected Design:



Analysis:

A backup is useful only when it represents a valid and sufficiently recent recovery point. If backups are performed irregularly or replication is delayed, the organization may lose recent updates.

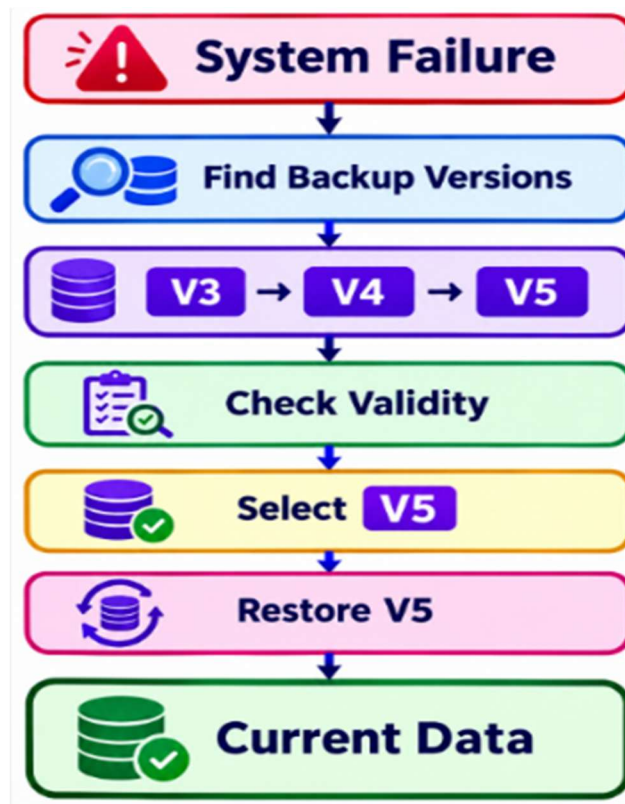
The problem can also occur when the system restores the **first available backup** instead of selecting the latest verified backup.

Corrective Actions:

- Perform backups at regular intervals.
- Use incremental or differential backups.
- Maintain multiple backup versions.

- Store backup timestamps.
- Verify successful backup completion.
- Replicate backups to another storage location.
- Regularly test restoration.
- Automatically select the latest valid recovery point.

Expected Recovery Process:



aFinal Solution:

The backup system should use versioned, regularly updated, and verified backups so that recovery always selects the latest valid copy.